UNIVERSITÉ ALIOUNE DIOP DE BAMBEY
UFR Sciences & Technologies | Master 2 Big Data & Intelligence Artificielle
Année universitaire 2025-2026

PROJET TUTORÉ — CODES SOURCES COMPLETS

Analyse de Sentiment et Tendances

Vox-SN

Services Publics & Fintech

Durée	4 semaines
Équipe	2 étudiants
Domaine	NLP, Sentiment Analysis & Fintech Social Listening
Standard	M2 Gouvernance Digitale / Approche Industrie
Enseignant	Mr Ahmed Ben Sidy Bouya SEYE — Senior Big Data & AI Engineer | Groupe Sonatel

NiFi • Kafka • Spark NLP • Pandera • HBase • Hive • Airflow • TF-IDF • Wolof/Français

1. Contexte et Objectifs

L'écoute citoyenne au Sénégal inclut désormais les services essentiels de la Fintech. Ce projet simule la collecte et l'analyse de posts concernant les services publics et les solutions de Mobile Money (Wave, Orange Money, Free Money), afin de mesurer la satisfaction client et détecter les crises de service en temps réel.

> ℹ️ **Enjeu Fintech & Gouvernance**
> Le Sénégal compte plus de 10 millions d'utilisateurs de Mobile Money. Une panne de Wave ou Orange Money génère des milliers de plaintes en quelques minutes sur les réseaux sociaux. Ce projet construit un système de veille automatique capable de déclencher des alertes avant qu'une crise ne dégénère.

1.1 Objectifs Techniques

#	Composant	Description
1	**Multi-Channel Ingestion**	Flux textuels via Apache NiFi (réseaux sociaux, réclamations, WhatsApp)
2	**Qualité & Contrat**	Validation Pandera + détection de PII (numéros de téléphone, transactions)
3	**NLP Pipeline**	Nettoyage Stopwords Wolof/FR, tokenisation, scoring sentiment lexical
4	**Privacy by Design**	Anonymisation des numéros de téléphone, user_id et IDs transactionnels
5	**Analytics & Mirroring**	HBase (alertes fraudes/pannes temps réel) + Hive (parts de voix, Battle MM)

2. Architecture de la Solution

```
[ SOURCES ]  ──▶  [ NiFi ]  ──▶  [ KAFKA ]  ──▶  [ SPARK NLP ]  ──▶  [ STORAGE ]
Posts/Fintech      Orchest.       Topic:           Nettoyage,          HBase / HDFS
Simulateur         & Routage       social_raw       Score Sentiment,         │
(FR/WO/EN)         par service     social_analyzed  Classification           ▼
                                                          │            [ ANALYTICS ]
                                                          ▼              Apache Hive
                                          [ ALERT LAYER ]            vue_battle_mm
                                         Crises & Fraudes            vue_parts_voix
                                         (score < -0.5)              vue_alertes
```

2.1 Stack Technologique

Couche	Technologie	Rôle
Ingestion	Apache NiFi 1.23	Collecte multi-canaux, filtrage par opérateur, routage vers Kafka
Message Broker	Apache Kafka 3.3	Topics : social_raw, social_analyzed, social_sentiment_agg
NLP Processing	Spark NLP 5.1.4	Tokenisation, suppression stopwords FR/Wolof, TF-IDF, scoring

Classification ML	Spark MLlib	Logistic Regression (sentiment) + Random Forest (catégorie)
Stockage	HBase 2.1 + HDFS	Alertes temps réel + modèles ML persistants
Analytics	Apache Hive 3.1	Vues Battle Mobile Money, parts de voix, détection crises
Validation	Pandera + regex PII	Schéma strict + détection numéros sénégalais (+221xxxxxxxxx)
MLOps	Airflow 2.7 + MLflow	Recalcul sentiment horaire + réentraînement conditionnel

3. Prérequis & Installation

> ℹ️ **Configuration minimale**
> RAM : 16 Go. CPU : 4 cœurs. OS : Ubuntu 22.04 LTS. Docker Engine 24.x. Java 11 requis par Spark NLP.

📄 **setup.sh**

```
# Vérification environnement
java -version       # Attendu : openjdk 11.x
docker --version    # Docker 24.x minimum
free -h             # Minimum 14 Go disponibles

# Installation dépendances
python3.9 -m venv venv_vox && source venv_vox/bin/activate
pip install -r requirements.txt

# Démarrage infrastructure
mkdir -p data/posts nifi_templates dags models
docker compose up -d zookeeper && sleep 10
docker compose up -d kafka nifi hbase hive-metastore && sleep 30
docker compose up -d spark-master spark-worker airflow

# Initialiser HBase (une seule fois)
python hbase_setup.py

# Interfaces web :
#   NiFi    : http://localhost:8081
#   Spark   : http://localhost:8080
#   HBase   : http://localhost:16010
#   Airflow : http://localhost:8082
```

📄 **requirements.txt**

```
# requirements.txt — Vox-SN
# Testé avec Python 3.9 / Java 11 / Spark 3.3

# Core pipeline
pyspark==3.3.2
kafka-python==2.0.2

# NLP
spark-nlp==5.1.4            # John Snow Labs — modèles FR/multilingual
nltk==3.8.1
langdetect==1.0.9

# Validation
pandera==0.17.2
pandas==2.0.3

# MLOps
apache-airflow==2.7.3
mlflow==2.8.1

# Stockage
pyhive==0.7.0
thrift==0.16.0
happybase==1.2.0

# Visualisation
matplotlib==3.7.3
seaborn==0.12.2
wordcloud==1.9.3

# Installation :
# pip install -r requirements.txt
# Java 11 requis : sudo apt install openjdk-11-jdk
```

```
# spark-nlp nécessite pyspark avec Java 11
```

4. Codes Sources Complets & Fonctionnels

4.1 Infrastructure Docker — docker-compose.yml

Infrastructure complète avec variable CITIZEN_SECRET_SALT injectée dans Spark, NiFi avec montage du dossier de posts, et réseau dédié vox_sn_net pour l'isolation des services.

docker-compose.yml

```yaml
version: '3.8'

services:
  zookeeper:
    image: bitnami/zookeeper:3.8
    container_name: zookeeper
    environment:
      ALLOW_ANONYMOUS_LOGIN: "yes"
    networks: [vox_sn_net]

  kafka:
    image: bitnami/kafka:3.3
    container_name: kafka
    depends_on: [zookeeper]
    ports: ["9092:9092"]
    environment:
      KAFKA_CFG_ZOOKEEPER_CONNECT: zookeeper:2181
      KAFKA_CFG_ADVERTISED_LISTENERS: PLAINTEXT://kafka:9092
      ALLOW_ANONYMOUS_LOGIN: "yes"
      KAFKA_CFG_AUTO_CREATE_TOPICS_ENABLE: "true"
    networks: [vox_sn_net]

  nifi:
    image: apache/nifi:1.23.2
    container_name: nifi
    ports: ["8081:8080"]
    volumes:
      - ./nifi_templates:/opt/nifi/nifi-current/conf/templates
      - ./data/posts:/opt/nifi/data/posts
    networks: [vox_sn_net]

  spark-master:
    image: bitnami/spark:3.3
    container_name: spark-master
    environment:
      SPARK_MODE: master
      CITIZEN_SECRET_SALT: UADB_VOX_2025
      JAVA_HOME: /opt/bitnami/java
    ports: ["8080:8080", "7077:7077"]
    volumes:
      - ./models:/opt/models
    networks: [vox_sn_net]

  spark-worker:
    image: bitnami/spark:3.3
    container_name: spark-worker
    depends_on: [spark-master]
    environment:
      SPARK_MODE: worker
      SPARK_MASTER_URL: spark://spark-master:7077
      CITIZEN_SECRET_SALT: UADB_VOX_2025
    networks: [vox_sn_net]

  hbase:
    image: harisekhon/hbase:2.1
    container_name: hbase
    ports: ["16010:16010", "9090:9090"]
    volumes:
      - hbase_data:/hbase-data
    networks: [vox_sn_net]
```

```
  hive-metastore:
    image: apache/hive:3.1.3
    container_name: hive-metastore
    environment:
      SERVICE_NAME: metastore
    networks: [vox_sn_net]

  airflow:
    image: apache/airflow:2.7.3
    container_name: airflow
    ports: ["8082:8080"]
    environment:
      AIRFLOW__CORE__EXECUTOR: LocalExecutor
      AIRFLOW__DATABASE__SQL_ALCHEMY_CONN: sqlite:////opt/airflow/airflow.db
    volumes:
      - ./dags:/opt/airflow/dags
      - ./models:/opt/airflow/models
    networks: [vox_sn_net]

volumes:
  hbase_data:
    driver: local

networks:
  vox_sn_net:
    driver: bridge
```

4.2 Initialisation HBase — hbase_setup.py

> ⚠️ **Fichier obligatoire**
> Sans ce fichier, le pipeline Spark ne peut pas écrire les alertes dans HBase. Crée les tables vox:posts, vox:alertes et vox:sentiment_agg. À exécuter une seule fois.

hbase_setup.py

```
# hbase_setup.py — Tables HBase pour Vox-SN
# Exécuter une seule fois après docker compose up

import happybase, logging
logger = logging.getLogger('HBaseSetup')

def create_vox_tables():
    conn = happybase.Connection('hbase', port=9090, timeout=10000)
    conn.open()
    tables = {
        # Posts analysés avec score sentiment
        b'vox:posts': {
            b'meta':      {'max_versions': 1},  # post_id, service, langue
            b'nlp':       {'max_versions': 1},  # sentiment_score, categorie, mots_cles
            b'privacy':   {'max_versions': 1},  # citizen_id_secure (jamais user_id brut)
        },
        # Alertes crises et fraudes (TTL 72h)
        b'vox:alertes': {
            b'alerte':    {'max_versions': 1, 'time_to_live': 259200},
        },
        # Agrégats sentiment par opérateur (rafraîchis par Airflow)
        b'vox:sentiment_agg': {
            b'stats':     {'max_versions': 48},  # 48h d'historique
        },
    }
    existantes = [t.decode() for t in conn.tables()]
    for nom_b, fam in tables.items():
        nom = nom_b.decode()
        if nom not in existantes:
            conn.create_table(nom_b, fam)
            logger.info(f'Table {nom} créée')
        else:
            logger.info(f'Table {nom} déjà existante')
    conn.close()

if __name__ == '__main__':
    logging.basicConfig(level=logging.INFO)
    create_vox_tables()
```

4.3 Contrats Pandera + Détection PII — schema.py

> ✓ **Spécificité Vox-SN**
> Le schéma inclut une validation active qui détecte et rejette les posts contenant des numéros de téléphone sénégalais (+221xxxxxxxxx, 7xxxxxxxx) ou des numéros de transaction. Un post contenant un PII brut est rejeté avant même d'entrer dans le pipeline.

schema.py

```
# schema.py — Contrats Pandera pour Vox-SN

import pandera as pa
from pandera.typing import Series
import pandas as pd, logging, re
```

```
logger = logging.getLogger('VoxSchema')

# Regex de détection des données sensibles (numéros WAVE, OM, téléphone)
PII_PATTERNS = [
    r'\+?221[0-9]{9}',           # Numéro sénégalais
    r'\b7[0-9]{8}\b',            # Mobile local
    r'\b[0-9]{10,16}\b',         # Numéro de compte/transaction
]

def contains_pii(text: str) -> bool:
    return any(re.search(p, text) for p in PII_PATTERNS)

class SocialSentimentSchema(pa.SchemaModel):
    post_id:       Series[str] = pa.Field(unique=True)
    service_cible: Series[str] = pa.Field(isin=[
        'SENELEC','SEN_EAU','TER','WAVE','ORANGE_MONEY','FREE_MONEY'
    ])
    texte_du_post: Series[str] = pa.Field(str_length={'min_value':5,'max_value':280})
    langue:        Series[str] = pa.Field(isin=['FR','WO','EN'])
    timestamp:     Series[str] = pa.Field(str_matches=r'^\d{4}-\d{2}-
\d{2}T\d{2}:\d{2}:\d{2}Z$')
    canal:         Series[str] =
pa.Field(isin=['TWITTER','FACEBOOK','WHATSAPP','RECLAMATION'])

    # Vérification qu'aucun PII brut n'entre dans le pipeline
    @pa.check('texte_du_post', name='no_raw_phone_in_text')
    def check_no_pii(cls, series):
        flagged = series.apply(contains_pii)
        if flagged.any():
            logger.warning(f'{flagged.sum()} post(s) contiennent des PII détectés')
        return ~flagged  # Rejeter les posts avec PII bruts

    class Config:
        strict = True
        coerce = True

def validate_and_filter(df: pd.DataFrame, schema_class) -> pd.DataFrame:
    try:
        return schema_class.validate(df, lazy=True)
    except pa.errors.SchemaErrors as exc:
        err = exc.failure_cases
        logger.warning(f'[{schema_class.__name__}] {len(err)} ligne(s) rejetée(s)')
        valid_idx = df.index.difference(err['index'].dropna().astype(int))
        return df.loc[valid_idx]
```

4.4 Lexique Sentiment Wolof/Français — lexique_sn.py

> ⚙️ **Unicité de ce projet**
> Ce lexique est la valeur ajoutée différenciante de Vox-SN. Il couvre les termes Wolof spécifiques au Sénégal ("dafa teye" = ça ne marche pas, "cher na" = c'est cher) ainsi que les plaintes typiques des utilisateurs de Mobile Money en Afrique de l'Ouest.

📄 **lexique_sn.py**

```
# lexique_sn.py — Lexique de sentiment Wolof/Français pour Vox-SN
# Spécificités sénégalaises : termes Wolof + argot Fintech local

# ── Termes négatifs ──────────────────────────────────────────────────────
NEGATIF = {
    # Wolof
    'dafa teye':      -0.9,  # ça ne marche pas
    'cher na':        -0.7,  # c'est cher
    'dafa neka':      -0.8,  # c'est nul
    'problem bi':     -0.6,  # le problème
    'douma gënn':     -0.8,  # je n'arrive pas
```

```
    'xam dina tax':   -0.7,  # exaspération
    'guiss dafa':     -0.5,  # voir que c'est mauvais
    # Français local
    'arnaque':        -0.95,
    'escroquerie':    -0.95,
    'panne':          -0.8,
    'indisponible':   -0.8,
    'frais cachés':   -0.85,
    'trop cher':      -0.75,
    'remboursement':  -0.6,
    'bloqué':         -0.7,
    'impossible':     -0.65,
    'nul':            -0.7,
    'honte':          -0.8,
    'vol':            -0.9,
    'transaction échouée': -0.85,
    'argent perdu':   -1.0,
}

# ── Termes positifs ──────────────────────────────────────────────────────
POSITIF = {
    # Wolof
    'dafa baax':      +0.9,  # c'est bien
    'jaama':          +0.7,  # paix / satisfaction
    'rafet':          +0.8,  # beau / bien
    'soo cool':       +0.7,  # très cool
    'dafa yomb':      +0.8,  # c'est facile
    # Français local
    'rapide':         +0.7,
    'pratique':       +0.75,
    'gratuit':        +0.8,
    'simple':         +0.7,
    'merci':          +0.6,
    'excellent':      +0.9,
    'top':            +0.8,
    'fiable':         +0.85,
}

# ── Catégories de plaintes ───────────────────────────────────────────────
CATEGORIES = {
    'TARIF':          ['cher', 'frais', 'coût', 'prix', 'commission', 'cher na'],
    'TECHNIQUE':      ['panne', 'teye', 'bloqué', 'indisponible', 'lent', 'bug'],
    'FRAUDE':         ['arnaque', 'escroquerie', 'vol', 'argent perdu', 'scam'],
    'SERVICE_CLIENT': ['réponse', 'attente', 'support', 'aide', 'joindre'],
    'POSITIF':        list(POSITIF.keys()),
}

# ── Stopwords Wolof à filtrer ────────────────────────────────────────────
STOPWORDS_WOLOF = {
    'ak','bi','gi','yi','mi','ci','di','na','la','ma','da',
    'nga','bu','ni','si','fi','woon','doon','nit','ñi',
}

STOPWORDS_FR_FINTECH = {
    'bonjour','bonsoir','svp','stp','merci','ok','oui','non',
    'wave','orange','free','money','mobile','senegal','sn',
}

ALL_STOPWORDS = STOPWORDS_WOLOF | STOPWORDS_FR_FINTECH
```

4.5 Simulateur de Posts Citoyens — kafka_producer_vox.py

Génère des posts réalistes en français, wolof et anglais sur les 6 services ciblés. Chaque service a des probabilités de sentiment négatif calibrées sur la réalité du terrain (SENELEC : 65% négatif, WAVE : 35% négatif). Les données sensitives (user_id, phone_number) sont présentes dans le simulateur et seront supprimées par la Privacy Layer Spark.

kafka_producer_vox.py

```
# kafka_producer_vox.py — Simulateur de posts citoyens Vox-SN
# Génère des posts réalistes en FR/Wolof sur les services publics et Fintech

from kafka import KafkaProducer
import json, random, time, uuid
from datetime import datetime

SERVICES = ['SENELEC','SEN_EAU','TER','WAVE','ORANGE_MONEY','FREE_MONEY']
CANAUX   = ['TWITTER','FACEBOOK','WHATSAPP','RECLAMATION']
LANGUES  = ['FR','WO','EN']

# Templates de posts réalistes par service
TEMPLATES_POSTS = {
    'WAVE': [
        ('FR', 'Mon transfert Wave est bloqué depuis 2h, impossible de joindre le support
!'),
        ('WO', 'Wave dafa teye, duma gënn xaalis bi !'),  # Wave ne marche pas, je
n'arrive pas
        ('FR', 'Wave rapide et pratique, meilleur service Mobile Money du Sénégal'),
        ('FR', 'Frais Wave trop cher na ! Orange Money moins cher'),
        ('WO', 'Wave dafa baax, dafa yomb'),  # C'est bien, c'est facile
        ('FR', 'Transaction Wave échouée, argent débité sans confirmation'),
    ],
    'ORANGE_MONEY': [
        ('FR', 'Orange Money encore en panne ce matin, inacceptable !'),
        ('FR', 'Les frais cachés Orange Money, c est de l arnaque !'),
        ('WO', 'Orange Money cher na trop, dafa neka'),
        ('FR', 'Remboursement Orange Money jamais reçu après 3 semaines'),
        ('FR', 'Orange Money fiable pour les transferts internationaux'),
    ],
    'FREE_MONEY': [
        ('FR', 'Free Money nouveau service, encore beaucoup de bugs'),
        ('FR', 'Free Money gratuit pour les transferts entre abonnés Free'),
        ('WO', 'Free Money problem bi, duma gënn'),
        ('FR', 'Compte Free Money bloqué sans explication du support'),
    ],
    'SENELEC': [
        ('FR', 'Coupure Senelec depuis 6h à Pikine, quand est-ce que ça revient ?'),
        ('FR', 'Facture Senelec incompréhensible, frais anormaux ce mois'),
        ('WO', 'Senelec dafa teye, xam dina tax !'),  # panne, exaspérant
        ('FR', 'Application Senelec bien améliorée, paiement facile maintenant'),
    ],
    'SEN_EAU': [
        ('FR', 'Pas d eau depuis 3 jours à Guédiawaye, Sen Eau ne répond pas'),
        ('FR', 'Sen Eau pression faible depuis une semaine, scandaleux'),
        ('WO', 'Sen Eau problem bi, dafa neka'),
    ],
    'TER': [
        ('FR', 'TER en retard encore aujourd hui, infos en temps réel impossible'),
        ('FR', 'TER pratique et rapide entre Dakar et Diamniadio, top !'),
        ('WO', 'TER dafa baax, rafet'),
    ],
}

# Probabilité de post négatif par service (basé sur satisfaction réelle)
PROB_NEGATIF = {
    'WAVE': 0.35, 'ORANGE_MONEY': 0.55, 'FREE_MONEY': 0.60,
    'SENELEC': 0.65, 'SEN_EAU': 0.70, 'TER': 0.45,
}

producer = KafkaProducer(
    bootstrap_servers=['kafka:9092'],
```

```
    value_serializer=lambda v: json.dumps(v, ensure_ascii=False).encode('utf-8')
)

def gen_post() -> dict:
    service  = random.choice(SERVICES)
    templates = TEMPLATES_POSTS.get(service, [])
    if templates:
        langue, texte = random.choice(templates)
    else:
        langue, texte = 'FR', f'Problème avec {service}'
    return {
        'post_id':      str(uuid.uuid4()),
        'user_id':      f'USR_{uuid.uuid4().hex[:10].upper()}',  # Sera anonymisé
        'phone_number': f'7{random.randint(10000000,99999999)}', # Sera supprimé
        'service_cible':service,
        'texte_du_post':texte,
        'langue':       langue,
        'canal':        random.choice(CANAUX),
        'timestamp':    datetime.utcnow().strftime('%Y-%m-%dT%H:%M:%SZ'),
        'region':       random.choice(['DAKAR','THIES','KAOLACK',
                                       'SAINT_LOUIS','ZIGUINCHOR']),
    }

if __name__ == '__main__':
    print('Simulateur Vox-SN démarré...')
    while True:
        post = gen_post()
        producer.send('social_raw', post)
        print(f'→ [{post["service_cible"]}] [{post["langue"]}]
{post["texte_du_post"][:60]}...')
        producer.flush()
        time.sleep(2)
```

4.6 Pipeline NLP Spark Streaming — streaming_sentiment.py

> ⚠️ **Privacy critique**
> Ce pipeline traite des données financières sensibles. Le drop('user_id', 'phone_number') après anonymisation SHA-256 est non négociable. Les numéros de transaction Wave/Orange Money ne doivent jamais apparaître dans HBase ou Hive.

📄 **streaming_sentiment.py**

```
# streaming_sentiment.py — Pipeline NLP Spark Streaming Vox-SN
# Privacy + Nettoyage Wolof/FR + Scoring sentiment + Classification

import os
from pyspark.sql import SparkSession
from pyspark.sql.functions import (
    col, sha2, concat, lit, from_json, to_json, struct,
    lower, regexp_replace, trim, split, array_contains,
    when, current_timestamp, coalesce, udf, window
)
from pyspark.sql.types import (
    StructType, StructField, StringType, FloatType
)
from pyspark.sql.types import StringType as ST
import json

SALT    = os.environ.get('CITIZEN_SECRET_SALT', 'default_salt')
BROKERS = 'kafka:9092'

spark = SparkSession.builder \
    .appName('Vox_SN_NLP') \
    .config('spark.sql.shuffle.partitions', '4') \
    .config('spark.jars.packages',
            'org.apache.spark:spark-sql-kafka-0-10_2.12:3.3.0') \
    .getOrCreate()
spark.sparkContext.setLogLevel('WARN')

post_schema = StructType([
    StructField('post_id',       StringType(), True),
    StructField('user_id',       StringType(), True),
    StructField('phone_number',  StringType(), True),
    StructField('service_cible', StringType(), True),
    StructField('texte_du_post', StringType(), True),
    StructField('langue',        StringType(), True),
    StructField('canal',         StringType(), True),
    StructField('timestamp',     StringType(), True),
    StructField('region',        StringType(), True),
])

# ── Chargement des lexiques (broadcast) ──────────────────────────────
from lexique_sn import NEGATIF, POSITIF, CATEGORIES, ALL_STOPWORDS
bc_negatif   = spark.sparkContext.broadcast(NEGATIF)
bc_positif   = spark.sparkContext.broadcast(POSITIF)
bc_categories= spark.sparkContext.broadcast(CATEGORIES)
bc_stopwords = spark.sparkContext.broadcast(ALL_STOPWORDS)

# ── UDF : Score de sentiment lexical Wolof/FR ────────────────────────
@udf(returnType=FloatType())
def score_sentiment(texte: str) -> float:
    if not texte:
        return 0.0
    texte_lower = texte.lower()
    score = 0.0
    count = 0
    for terme, val in bc_negatif.value.items():
        if terme in texte_lower:
            score += val; count += 1
    for terme, val in bc_positif.value.items():
        if terme in texte_lower:
            score += val; count += 1
    return float(score / max(count, 1))
```

```
# ── UDF : Catégorisation de la plainte ──────────────────────────────────
@udf(returnType=StringType())
def categoriser(texte: str) -> str:
    if not texte:
        return 'INCONNU'
    texte_lower = texte.lower()
    scores = {}
    for cat, mots in bc_categories.value.items():
        scores[cat] = sum(1 for m in mots if m in texte_lower)
    best = max(scores, key=scores.get)
    return best if scores[best] > 0 else 'AUTRE'

# ── UDF : Nettoyage texte (Stopwords FR + Wolof) ────────────────────────
@udf(returnType=StringType())
def nettoyer_texte(texte: str) -> str:
    if not texte:
        return ''
    tokens = texte.lower().split()
    tokens = [t for t in tokens if t not in bc_stopwords.value
              and len(t) > 2 and t.isalpha()]
    return ' '.join(tokens)

# ── Lecture Kafka ───────────────────────────────────────────────────────
raw_df = (spark.readStream
    .format('kafka')
    .option('kafka.bootstrap.servers', BROKERS)
    .option('subscribe', 'social_raw')
    .option('startingOffsets', 'latest')
    .load()
    .select(from_json(col('value').cast('string'), post_schema).alias('d'))
    .select('d.*')
    .withColumn('event_ts', current_timestamp())
)

# ── Privacy Layer : suppression user_id et phone_number ─────────────────
# CORRECTION : drop des 2 champs PII (user_id ET phone_number)
secure_df = (raw_df
    .withColumn('citizen_id_secure',
                sha2(concat(col('user_id'), lit(SALT)), 256))
    .drop('user_id', 'phone_number')  # Suppression OBLIGATOIRE des 2 PII
)

# ── Nettoyage NLP ───────────────────────────────────────────────────────
# Étape 1 : normalisation
clean_df = (secure_df
    .withColumn('texte_norm',
        trim(regexp_replace(
            lower(col('texte_du_post')),
            r'[^a-zàáâäèéêëîïôùûüœç\s]', ' '  # Conserver accents FR
        ))
    )
)

# Étape 2 : application des UDFs NLP
nlp_df = (clean_df
    .withColumn('texte_clean',    nettoyer_texte(col('texte_norm')))
    .withColumn('sentiment_score',score_sentiment(col('texte_norm')))
    .withColumn('categorie',      categoriser(col('texte_norm')))
    .withColumn('sentiment_label',
        when(col('sentiment_score') < -0.5, lit('NEGATIF_FORT'))
        .when(col('sentiment_score') < 0.0, lit('NEGATIF'))
        .when(col('sentiment_score') > 0.3, lit('POSITIF'))
        .otherwise(lit('NEUTRE'))
    )
    .withColumn('statut_alerte',
        when(
            (col('sentiment_score') < -0.5) &
            (col('categorie').isin('FRAUDE','TECHNIQUE')),
            lit('CRISE')
        )
        .when(col('sentiment_score') < -0.5, lit('NEGATIF_FORT'))
        .otherwise(lit('NORMAL'))
```

```
    )
)

# ── Agrégation fenêtre glissante 1h par opérateur ──────────────────────────
sentiment_window = (nlp_df
    .withWatermark('event_ts', '10 minutes')
    .groupBy(window('event_ts', '1 hour', '15 minutes'), 'service_cible')
    .agg(
        {'sentiment_score': 'avg', 'post_id': 'count'}
    )
    .withColumnRenamed('avg(sentiment_score)', 'sentiment_moyen')
    .withColumnRenamed('count(post_id)',        'nb_posts')
    .withColumn('statut_operateur',
        when(col('sentiment_moyen') < -0.5, lit('CRISE'))
        .when(col('sentiment_moyen') < -0.2, lit('ATTENTION'))
        .otherwise(lit('NORMAL'))
    )
)

# ── Écriture : posts analysés vers Kafka ───────────────────────────────────
q1 = (nlp_df
    .select(to_json(struct('*')).alias('value'))
    .writeStream.format('kafka')
    .option('kafka.bootstrap.servers', BROKERS)
    .option('topic', 'social_analyzed')
    .option('checkpointLocation', '/tmp/vox_posts_ckpt')
    .outputMode('append').start()
)

# ── Écriture : agrégats sentiment par opérateur ────────────────────────────
q2 = (sentiment_window
    .select(to_json(struct('*')).alias('value'))
    .writeStream.format('kafka')
    .option('kafka.bootstrap.servers', BROKERS)
    .option('topic', 'social_sentiment_agg')
    .option('checkpointLocation', '/tmp/vox_agg_ckpt')
    .outputMode('update').start()
)

q1.awaitTermination()
```

4.7 Entraînement ML Multi-classe — train_classifier.py

Deux modèles entraînés en parallèle : (1) Logistic Regression pour la classification binaire du sentiment (Positif/Négatif/Neutre), (2) Random Forest pour la catégorisation des plaintes (Tarif/Technique/Fraude/Service Client). Pipeline TF-IDF avec StopWordsRemover configuré pour le Wolof et le français.

train_classifier.py

```
# train_classifier.py — Classification multi-classe Spark MLlib
# Sentiment (Positif/Négatif) + Catégorie (Tarif/Réseau/ServiceClient/Fraude)

from pyspark.sql import SparkSession
from pyspark.ml.feature    import Tokenizer, StopWordsRemover, HashingTF, IDF,
VectorAssembler
from pyspark.ml.classification import LogisticRegression, RandomForestClassifier
from pyspark.ml.evaluation import MulticlassClassificationEvaluator
from pyspark.ml            import Pipeline
from pyspark.ml.feature    import StringIndexer
import argparse

parser = argparse.ArgumentParser()
parser.add_argument('--output-path', default='hdfs:///vox_sn/models/classifier_latest')
parser.add_argument('--window-days', type=int, default=30)
args = parser.parse_args()

spark = SparkSession.builder \
    .appName('VoxSN_TrainClassifier') \
    .enableHiveSupport().getOrCreate()

# ── Données d'entraînement depuis Hive ──────────────────────────────────
df = spark.sql(f"""
    SELECT texte_clean,
           sentiment_label,  -- Variable cible 1 : Positif/Négatif/Neutre
           categorie         -- Variable cible 2 : Tarif/Technique/Fraude/...
    FROM vox_sn.posts_analyses
    WHERE date_post >= DATE_SUB(CURRENT_DATE, {args.window_days})
      AND texte_clean IS NOT NULL
      AND sentiment_label IS NOT NULL
""").na.drop()

train, test = df.randomSplit([0.8, 0.2], seed=42)
print(f'Train={train.count()} | Test={test.count()}')

# ── Pipeline ML — TF-IDF + Logistic Regression ──────────────────────────
# Modèle 1 : classification du sentiment
label_idx_sent  = StringIndexer(inputCol='sentiment_label', outputCol='label_sent')
tokenizer       = Tokenizer(inputCol='texte_clean', outputCol='tokens')
remover         = StopWordsRemover(inputCol='tokens', outputCol='tokens_clean')
hashing_tf      = HashingTF(inputCol='tokens_clean', outputCol='tf', numFeatures=5000)
idf             = IDF(inputCol='tf', outputCol='features')
lr_sent         = LogisticRegression(featuresCol='features', labelCol='label_sent',
                                     maxIter=20, regParam=0.01)

pipeline_sent = Pipeline(stages=[label_idx_sent, tokenizer, remover,
                                 hashing_tf, idf, lr_sent])
model_sent = pipeline_sent.fit(train)

# ── Modèle 2 : classification des catégories ────────────────────────────
label_idx_cat  = StringIndexer(inputCol='categorie', outputCol='label_cat')
rf_cat         = RandomForestClassifier(featuresCol='features', labelCol='label_cat',
                                        numTrees=100, maxDepth=6, seed=42)

pipeline_cat = Pipeline(stages=[label_idx_cat, tokenizer, remover,
                                hashing_tf, idf, rf_cat])
model_cat = pipeline_cat.fit(train)

# ── Évaluation ──────────────────────────────────────────────────────────
ev = MulticlassClassificationEvaluator(metricName='f1')
```

```
preds_sent = model_sent.transform(test)
ev.setLabelCol('label_sent').setPredictionCol('prediction')
print(f'[Sentiment] F1 = {ev.evaluate(preds_sent):.4f}')

preds_cat = model_cat.transform(test)
ev.setLabelCol('label_cat').setPredictionCol('prediction')
print(f'[Catégorie] F1 = {ev.evaluate(preds_cat):.4f}')

# ── Sauvegarde ──────────────────────────────────────────────────────────
model_sent.write().overwrite().save(f'{args.output_path}/sentiment')
model_cat .write().overwrite().save(f'{args.output_path}/categorie')
print(f'Modèles sauvegardés dans {args.output_path}')
spark.stop()
```

4.8 DAG Airflow de Monitoring — vox_sn_monitoring_dag.py

DAG déclenché chaque heure (schedule_interval='0 * * * *'). Le BranchPythonOperator détecte les crises actives : si plus de 3 opérateurs sont en état CRISE simultanément, le réentraînement du modèle NLP est déclenché automatiquement.

dags/vox_sn_monitoring_dag.py

```
# dags/vox_sn_monitoring_dag.py — DAG Airflow Vox-SN
# Recalcul sentiment toutes les heures + réentraînement hebdomadaire

from airflow import DAG
from airflow.operators.python import PythonOperator, BranchPythonOperator
from airflow.operators.dummy  import DummyOperator
from airflow.utils.dates      import days_ago
from datetime import timedelta
import subprocess, logging

logger = logging.getLogger('vox_dag')
default_args = {
    'owner':'seye_ahmed', 'retries':2,
    'retry_delay':timedelta(minutes=5),
    'email_on_failure':True, 'email':['vox-sn@sonatel.sn']
}

# ── T1 : Recalcul sentiment par opérateur (toutes les heures) ────────────
def recalculate_sentiment(**ctx):
    from pyhive import hive
    conn = hive.Connection(host='hive-metastore', port=10000)
    cursor = conn.cursor()
    cursor.execute("""
        INSERT OVERWRITE TABLE vox_sn.sentiment_hourly
        SELECT
            service_cible,
            DATE_TRUNC('hour', CURRENT_TIMESTAMP) AS heure,
            COUNT(*)                              AS nb_posts,
            AVG(sentiment_score)                  AS sentiment_moyen,
            SUM(CASE WHEN categorie='FRAUDE'    THEN 1 ELSE 0 END) AS nb_fraudes,
            SUM(CASE WHEN categorie='TECHNIQUE' THEN 1 ELSE 0 END) AS nb_pannes,
            CASE
                WHEN AVG(sentiment_score) < -0.5 THEN 'CRISE'
                WHEN AVG(sentiment_score) < -0.2 THEN 'ATTENTION'
                ELSE 'NORMAL'
            END AS statut
        FROM vox_sn.posts_analyses
        WHERE date_post >= DATE_SUB(CURRENT_DATE, 1)
        GROUP BY service_cible
    """)
    logger.info('Sentiment horaire recalculé')

# ── T2 : Détection et alerte de crise ────────────────────────────────────
def detect_crises(**ctx):
```

```
    from pyhive import hive
    conn = hive.Connection(host='hive-metastore', port=10000)
    cursor = conn.cursor()
    cursor.execute("""
        SELECT service_cible, sentiment_moyen, nb_fraudes, statut
        FROM vox_sn.sentiment_hourly
        WHERE statut IN ('CRISE','ATTENTION')
          AND heure >= DATE_SUB(CURRENT_TIMESTAMP, 1/24.0)
    """)
    crises = cursor.fetchall()
    if crises:
        for row in crises:
            logger.warning(f'CRISE DÉTECTÉE : {row[0]} | score={row[1]:.2f}')
        ctx['ti'].xcom_push(key='nb_crises', value=len(crises))
    return 'trigger_retrain' if len(crises) > 3 else 'skip_retrain'

# ── T3 : Réentraînement si trop de mauvaises classifications ──────────────
def trigger_retrain(**ctx):
    result = subprocess.run([
        'spark-submit', '--master', 'spark://spark-master:7077',
        '/opt/airflow/models/train_classifier.py',
        '--output-path', 'hdfs:///vox_sn/models/classifier_latest',
        '--window-days', '30'
    ], capture_output=True, text=True, timeout=3600)
    if result.returncode != 0:
        raise Exception(f'Spark job failed: {result.stderr}')
    logger.info('Modèle NLP mis à jour')

with DAG('vox_sn_monitoring',
         default_args=default_args,
         description='Monitoring sentiment Vox-SN — recalcul horaire',
         schedule_interval='0 * * * *',  # Chaque heure
         start_date=days_ago(1), catchup=False,
         tags=['vox-sn','nlp','fintech']) as dag:

    start  = DummyOperator(task_id='start')
    skip   = DummyOperator(task_id='skip_retrain')
    end    = DummyOperator(task_id='end')

    t1 = PythonOperator(task_id='recalculate_sentiment',
                        python_callable=recalculate_sentiment, provide_context=True)
    branch = BranchPythonOperator(task_id='detect_crises',
                                  python_callable=detect_crises, provide_context=True)
    t3 = PythonOperator(task_id='trigger_retrain',
                        python_callable=trigger_retrain, provide_context=True)

    start >> t1 >> branch >> [t3, skip]
    t3 >> end; skip >> end
```

4.9 Tables Hive & Vues Analytics — hive_setup.sql

Trois vues essentielles : vue_battle_mobile_money pour la comparaison Wave/OM/Free sur 7 jours, vue_parts_de_voix pour les parts de mention par service sur 30 jours, et vue_alertes_crises qui détecte automatiquement les crises confirmées (minimum 5 posts négatifs sur 1h).

hive_setup.sql

```
-- hive_setup.sql — Tables et vues Hive Vox-SN
-- Parts de voix, tendances sentiment, comparaison opérateurs

CREATE DATABASE IF NOT EXISTS vox_sn
    COMMENT 'Analyse sentiment services publics et Fintech — UADB 2025';
USE vox_sn;

-- Table principale des posts analysés
CREATE TABLE IF NOT EXISTS posts_analyses (
    post_id         STRING,
    citizen_id_secure STRING COMMENT 'SHA-256 — jamais user_id brut',
    service_cible   STRING,
    texte_clean     STRING,
    langue          STRING,
    canal           STRING,
    region          STRING,
    sentiment_score FLOAT,
    sentiment_label STRING,
    categorie       STRING,
    statut_alerte   STRING,
    ingestion_ts    TIMESTAMP
)
PARTITIONED BY (date_post STRING, service STRING)
STORED AS ORC TBLPROPERTIES ('orc.compress'='SNAPPY');

-- Table agrégats horaires (alimentée par Airflow)
CREATE TABLE IF NOT EXISTS sentiment_hourly (
    service_cible   STRING,
    heure           TIMESTAMP,
    nb_posts        INT,
    sentiment_moyen FLOAT,
    nb_fraudes      INT,
    nb_pannes       INT,
    statut          STRING
) STORED AS ORC;

-- Vue : Battle des Mobile Money (Wave vs OM vs Free)
CREATE OR REPLACE VIEW vue_battle_mobile_money AS
SELECT
    service_cible,
    COUNT(*)                                AS total_mentions,
    AVG(COALESCE(sentiment_score, 0))       AS sentiment_moyen,
    SUM(CASE WHEN sentiment_label='POSITIF'   THEN 1 ELSE 0 END) * 100.0
        / COUNT(*)                           AS pct_positif,
    SUM(CASE WHEN sentiment_label='NEGATIF_FORT' THEN 1 ELSE 0 END) * 100.0
        / COUNT(*)                           AS pct_critique,
    SUM(CASE WHEN categorie='FRAUDE'        THEN 1 ELSE 0 END) AS nb_fraudes,
    SUM(CASE WHEN categorie='TARIF'         THEN 1 ELSE 0 END) AS nb_plaintes_tarif,
    SUM(CASE WHEN categorie='TECHNIQUE'     THEN 1 ELSE 0 END) AS nb_pannes
FROM posts_analyses
WHERE service_cible IN ('WAVE','ORANGE_MONEY','FREE_MONEY')
  AND date_post >= DATE_SUB(CURRENT_DATE(), 7)
GROUP BY service_cible
ORDER BY sentiment_moyen DESC;

-- Vue : Parts de voix par service (tous)
CREATE OR REPLACE VIEW vue_parts_de_voix AS
SELECT
    service_cible,
    COUNT(*) AS mentions,
    COUNT(*) * 100.0 / SUM(COUNT(*)) OVER () AS part_de_voix_pct,
    AVG(COALESCE(sentiment_score, 0))       AS nps_approx
FROM posts_analyses
```

```
WHERE date_post >= DATE_SUB(CURRENT_DATE(), 30)
GROUP BY service_cible
ORDER BY mentions DESC;

-- Vue : Détection de crise (sentiment < -0.5 sur fenêtre 1h)
CREATE OR REPLACE VIEW vue_alertes_crises AS
SELECT
    service_cible,
    categorie,
    COUNT(*)                          AS nb_posts_negatifs,
    AVG(COALESCE(sentiment_score, 0)) AS sentiment_moyen,
    MIN(ingestion_ts)                 AS debut_crise,
    MAX(ingestion_ts)                 AS derniere_maj
FROM posts_analyses
WHERE statut_alerte = 'CRISE'
  AND ingestion_ts >= DATE_SUB(CURRENT_TIMESTAMP, 1/24.0)
GROUP BY service_cible, categorie
HAVING COUNT(*) >= 5  -- Min 5 posts négatifs = crise confirmée
ORDER BY nb_posts_negatifs DESC;
```

4.10 Rapport de Crise — rapport_crise.py

Livrable 4 : génère un rapport graphique en 4 panneaux illustrant une panne Wave — timeline du sentiment, distribution des catégories de plaintes, Battle Mobile Money comparatif, et nuage de mots des termes négatifs en Wolof/Français.

> ✓ **Résultat attendu**
> Le rapport doit montrer une chute du sentiment Wave sous -0.5 pendant la période de panne, un pic de plaintes catégorie TECHNIQUE, et un avantage concurrentiel temporaire pour les concurrents (Orange Money, Free Money).

rapport_crise.py

```
# rapport_crise.py — Simulation d'une panne Mobile Money + génération rapport
# Livrable 4 : Rapport de crise scénario panne Wave

import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.dates as mdates
import seaborn as sns
from wordcloud import WordCloud
from pyhive import hive
from datetime import datetime, timedelta

conn = hive.Connection(host='hive-metastore', port=10000, database='vox_sn')

# ── 1. Charger les données de crise ──────────────────────────────────────
crisis_df = pd.read_sql("""
    SELECT ingestion_ts, service_cible, sentiment_score,
           categorie, texte_clean, langue, region
    FROM posts_analyses
    WHERE service_cible = 'WAVE'
      AND statut_alerte IN ('CRISE','NEGATIF_FORT')
    ORDER BY ingestion_ts
""", conn)

crisis_df['ingestion_ts'] = pd.to_datetime(crisis_df['ingestion_ts'])

# ── 2. Timeline de l'évolution du sentiment ──────────────────────────────
fig, axes = plt.subplots(2, 2, figsize=(16, 10))
fig.suptitle('Rapport de Crise — Panne Wave Sénégal | Vox-SN', fontsize=16,
fontweight='bold')
```

```
# Graphique 1 : Timeline sentiment Wave
timeline = crisis_df.set_index('ingestion_ts')['sentiment_score'].resample('15min').mean()
axes[0,0].plot(timeline.index, timeline.values, color='#E53935', linewidth=2)
axes[0,0].axhline(y=-0.5, color='black', linestyle='--', label='Seuil crise (-0.5)')
axes[0,0].fill_between(timeline.index, timeline.values, -0.5,
                       where=timeline.values<-0.5, alpha=0.3, color='red')
axes[0,0].set_title('Évolution sentiment Wave (fenêtres 15min)')
axes[0,0].set_ylabel('Score sentiment')
axes[0,0].legend()
axes[0,0].xaxis.set_major_formatter(mdates.DateFormatter('%H:%M'))

# Graphique 2 : Distribution des catégories
cat_counts = crisis_df['categorie'].value_counts()
axes[0,1].barh(cat_counts.index, cat_counts.values,
               color=['#E53935','#FF7043','#5E35B1','#00695C','#F57F17'])
axes[0,1].set_title('Catégories de plaintes durant la crise')
axes[0,1].set_xlabel('Nombre de posts')

# Graphique 3 : Comparaison Battle Mobile Money
battle_df = pd.read_sql("SELECT * FROM vue_battle_mobile_money", conn)
if not battle_df.empty:
    x = range(len(battle_df))
    axes[1,0].bar(x, battle_df['sentiment_moyen'],
                  color=['#E53935' if v<0 else '#00695C'
                         for v in battle_df['sentiment_moyen']])
    axes[1,0].set_xticks(list(x))
    axes[1,0].set_xticklabels(battle_df['service_cible'])
    axes[1,0].set_title('Battle Mobile Money — Score sentiment moyen (7 jours)')
    axes[1,0].set_ylabel('Score sentiment')
    axes[1,0].axhline(y=0, color='black', linewidth=0.5)

# Graphique 4 : Nuage de mots (posts négatifs Wave)
textes_negatifs = ' '.join(crisis_df['texte_clean'].dropna().tolist())
if textes_negatifs.strip():
    wc = WordCloud(width=600, height=300, background_color='white',
                   colormap='Reds', max_words=50).generate(textes_negatifs)
    axes[1,1].imshow(wc, interpolation='bilinear')
    axes[1,1].axis('off')
    axes[1,1].set_title('Mots-clés des plaintes Wave (Wolof + FR)')

plt.tight_layout()
plt.savefig('rapport_crise_wave.png', dpi=150, bbox_inches='tight')
print('Rapport de crise généré : rapport_crise_wave.png')

# ── 3. Métriques clés de la crise ──────────────────────────────────────
print('\n=== MÉTRIQUES DE CRISE ===')
print(f'Total posts négatifs   : {len(crisis_df)}')
print(f'Score sentiment moyen  : {crisis_df["sentiment_score"].mean():.3f}')
print(f'% plaintes Fraude      : {(crisis_df["categorie"]=="FRAUDE").mean()*100:.1f}%')
print(f'% plaintes Technique   : {(crisis_df["categorie"]=="TECHNIQUE").mean()*100:.1f}%')
if not crisis_df.empty:
    debut = crisis_df['ingestion_ts'].min()
    fin   = crisis_df['ingestion_ts'].max()
    duree = fin - debut
    print(f'Durée de la crise      : {duree}')
    print(f'Langues : {crisis_df["langue"].value_counts().to_dict()}')
```

5. Guide de Test

5.1 Tester le pipeline end-to-end

test_pipeline.sh

```
# 1. Vérifier les topics Kafka
docker exec kafka kafka-topics.sh \
  --bootstrap-server localhost:9092 --list
# Attendu : social_raw, social_analyzed, social_sentiment_agg

# 2. Démarrer le simulateur
python kafka_producer_vox.py &

# 3. Lire les posts bruts (avec PII)
docker exec kafka kafka-console-consumer.sh \
  --bootstrap-server localhost:9092 --topic social_raw \
  --from-beginning --max-messages 5

# 4. Lire les posts analysés (PII supprimés, sentiment scoré)
docker exec kafka kafka-console-consumer.sh \
  --bootstrap-server localhost:9092 --topic social_analyzed \
  --from-beginning --max-messages 5
# Vérifier : user_id ABSENT, citizen_id_secure PRÉSENT
# Vérifier : sentiment_score et categorie PRÉSENTS

# 5. Déclencher une alerte CRISE manuellement
# Dans kafka_producer_vox.py, envoyer 10 posts WAVE négatifs d'affilée :
# python -c "
# from kafka_producer_vox import producer
# import json
# for i in range(10):
#     producer.send('social_raw', {'service_cible':'WAVE',
#     'texte_du_post':'Wave dafa teye, arnaque total argent perdu !',
#     'langue':'WO', ...})
# producer.flush()"
```

5.2 Vérifier les vues Hive

test_hive.sh

```
# Vérifier la Battle Mobile Money
docker exec hive-metastore beeline -u jdbc:hive2://localhost:10000 \
  -e 'SELECT * FROM vox_sn.vue_battle_mobile_money;'

# Vérifier les alertes de crise
docker exec hive-metastore beeline -u jdbc:hive2://localhost:10000 \
  -e 'SELECT * FROM vox_sn.vue_alertes_crises;'

# Parts de voix
docker exec hive-metastore beeline -u jdbc:hive2://localhost:10000 \
  -e 'SELECT * FROM vox_sn.vue_parts_de_voix;'
```

6. Livrables Attendus

#	Livrable	Contenu attendu
1	**Template NiFi (.xml)**	Flux filtrant les messages par opérateur (RouteOnContent) avec 6 routes distinctes
2	**Pipeline NLP Spark**	streaming_sentiment.py complet avec UDFs Wolof/FR, scoring et classification fonctionnels
3	**Dashboard Battle Mobile Money**	Vue Hive + visualisation comparative Wave vs Orange Money vs Free Money (7 jours glissants)
4	**Rapport de Crise**	rapport_crise.py avec 4 graphiques illustrant une panne Wave et ses effets sur le sentiment

7. Barème de Notation — /20 points

> ℹ️ **Barème différencié 4 niveaux**
> Chaque critère est évalué selon 4 niveaux de maîtrise. Bonus documentation jusqu'à +2 pts. La démonstration live d'une alerte CRISE en soutenance est exigée.

Critère	Pts	Insuffisant (0)	Passable (50%)	Bien (75%)	Excellent (100%)
Ingestion NiFi / Kafka	3 pts	NiFi absent ou non fonctionnel	1 opérateur routé sur 6	3 opérateurs routés, topics créés	6 opérateurs séparés, messages bien formés dans social_raw
Validation & Privacy (Pandera + PII)	4 pts	Aucune validation, PII en clair	Schéma partiel sans détection PII	Schéma complet + SHA-256, user_id non supprimé	PII détecté par regex, drop(user_id, phone_number) confirmé dans HBase
NLP & Prétraitement Wolof/FR	4 pts	Pas de traitement NLP	Nettoyage basique FR seulement	Stopwords Wolof intégrés, scoring simple	Lexique Wolof/FR complet, UDFs fonctionnelles, catégorisation correcte
MLOps & Classification Spark ML	5 pts	Pas de modèle ML	Scoring lexical seul, pas de ML	1 modèle Logistic Regression fonctionnel	2 modèles (sentiment + catégorie) + DAG horaire + F1 > 0.7 démontré
Mirroring HBase/Hive & Dashboards	4 pts	Ni HBase ni Hive configurés	Tables créées mais vides	Vues Hive fonctionnelles, pas de dashboard	Battle Mobile Money live + alerte CRISE visible en soutenance
TOTAL	20 pts				

Bonus Documentation (jusqu'à +2 pts)

Élément	Points bonus
README.md avec docker compose up + captures d'écran	+0.5 pt
Commentaires de code en français + docstrings	+0.5 pt
Tests unitaires pytest sur les UDFs NLP et Pandera	+0.5 pt
Démo live alerte CRISE sans erreur en soutenance	+0.5 pt

⚠️ **Rappel Privacy Fintech**

Les données de Mobile Money (numéros de téléphone, montants de transactions, codes secrets) sont des données ultra-sensibles. Toute présence de PII brut dans HBase, Hive ou les logs Spark entraîne la perte intégrale des 4 pts Privacy, sans recours possible.

Bon courage à toutes les équipes !

"Always be a solution, never a problem."

Mr Ahmed Ben Sidy Bouya SEYE | Senior Big Data & AI Engineer | Groupe Sonatel